

Tree ADT

- Position as Node abstraction
 - Generic methods:
 - integer **size()**
 - boolean **isEmpty()**
 - Iterator **iterator()**
 - Iterable **positions()**
 - Access methods:
 - Position **root()**
 - Position **parent(p)**
 - Iterable **children(p)**
 - Integer **numChildren(p)**
- ▶ Query methods:
 - ▶ boolean **isInternal(p)**
 - ▶ boolean **isExternal(p)**
 - ▶ boolean **isRoot(p)**
 - ▶ Additional update methods may be defined by data structures implementing the Tree ADT

Node object

Node object implementation typically has the following attributes:

- value: the value associated with this Node
- children: set or list of children of this Node
- parent: (optional) the parent of this Node

```
def is_external(v)
    # test if v is a leaf
    return v.children.is_empty()
```

```
def is_root(v)
    # test if v is the root
    return v.parent == null
```

Traversing trees

A **traversal** visits the nodes of a tree in a systematic manner

When traversing a simpler structure like a list there is one natural traversal strategy (forward or backwards)

Trees are more complex and admit more than one natural way:

- pre-order
- post-order
- in-order (for binary trees)

Preorder Traversal

To do a preorder traversal starting at a given node, we visit the node before visiting its descendants

```
def pre_order(v)
    visit(v)
    for each child w of v
        pre_order(w)
```

If tree is ordered visit the child subtrees in the prescribed order

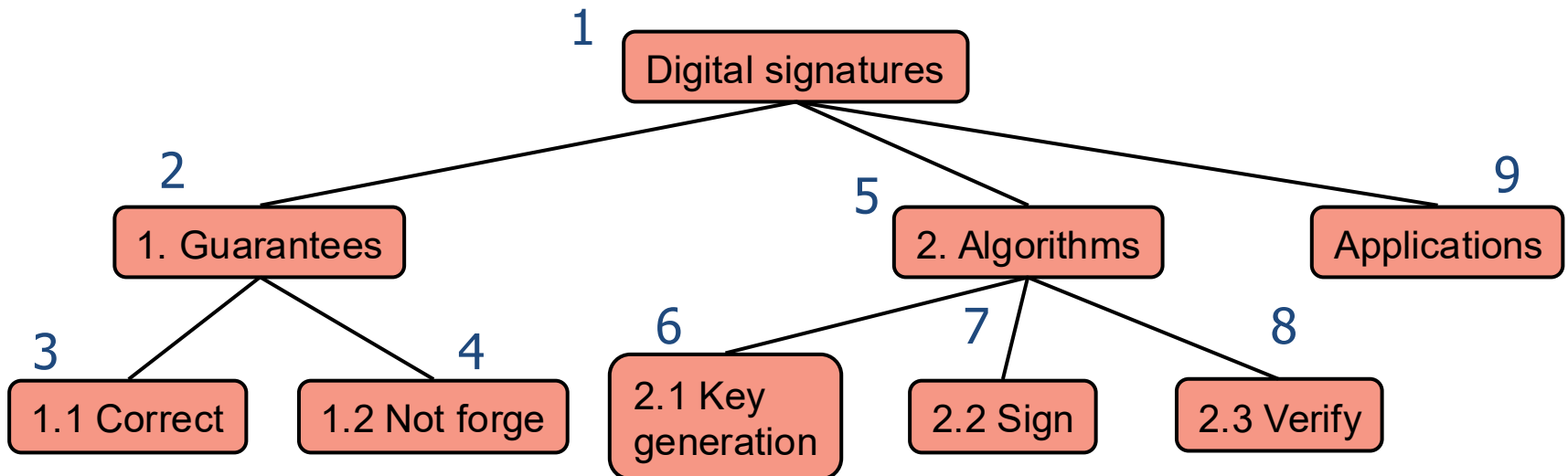
Visit does some work on the node:

- print node data
- aggregate node data
- modify node data

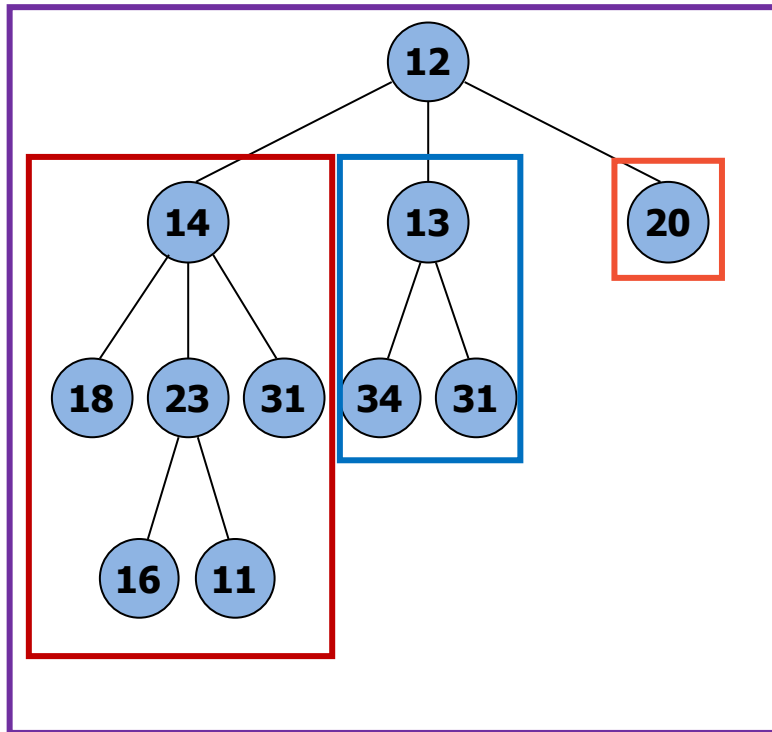
Preorder Traversal Example

Nodes are numbered in the order they are visited when we call `pre_order()` at the root

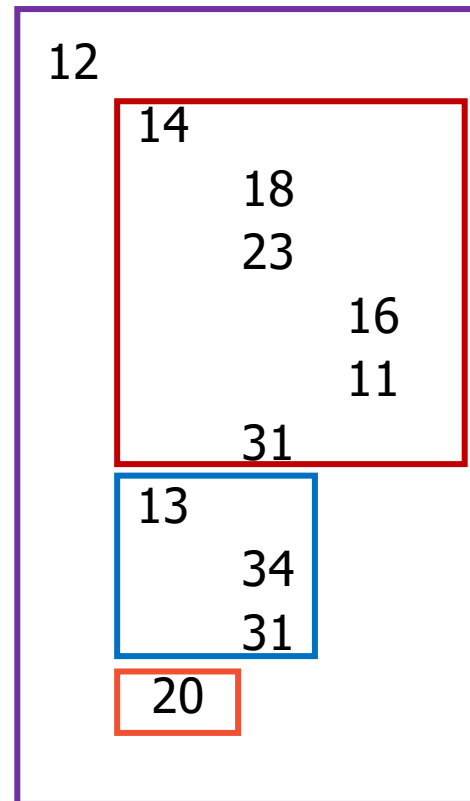
```
def pre_order(v)
    visit(v)
    for each child w of v
        pre_order(w)
```



Preorder Traversal Example



visit
order



Preorder
traversal
of subtree

Postorder Traversal

To do a postorder traversal starting at a given node, we visit the node after its descendants

```
def post_order(v)
    for each child w of v
        post_order(w)
    visit(v)
```

If tree is ordered visit the child subtrees in the prescribed order

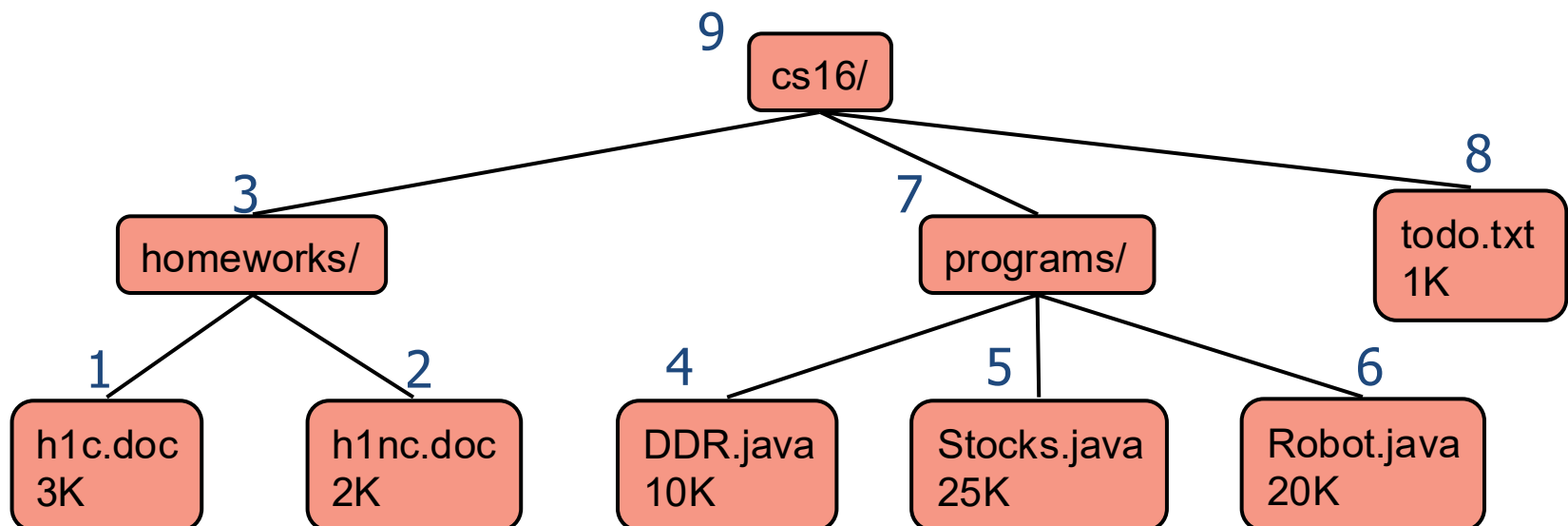
Visit does some work on the node:

- print node data
- aggregate node data
- modify node data

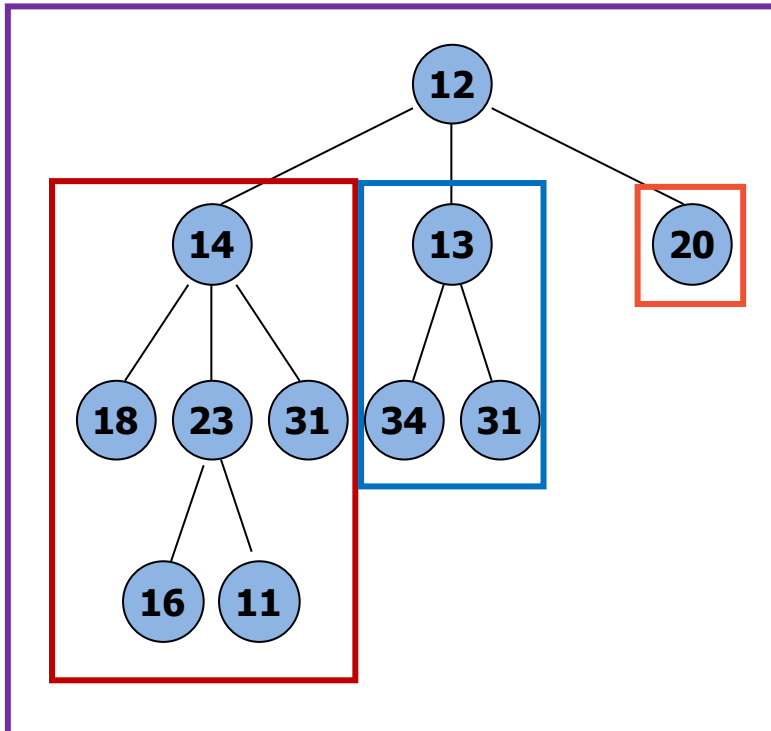
Postorder Traversal

Nodes are numbered in the order they are visited when we call `post_order()` at the root

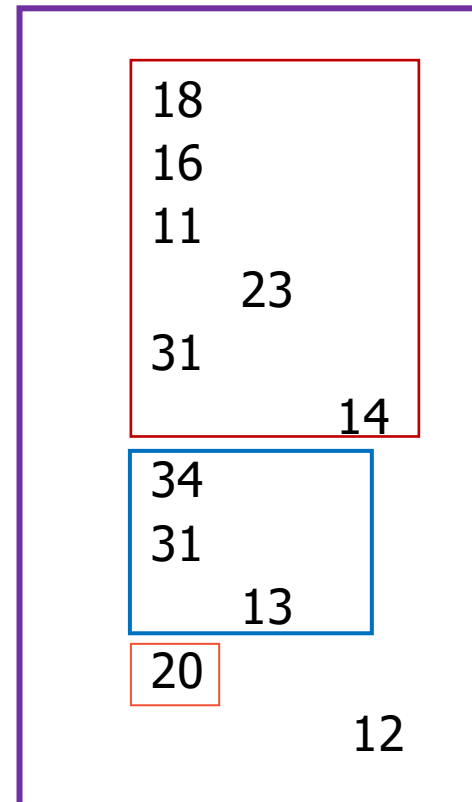
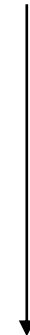
```
def post_order(v)
  for each child w of v
    post_order(w)
  visit(v)
```



Traversing in postorder



visit
order



Postorder
traversal
of subtree

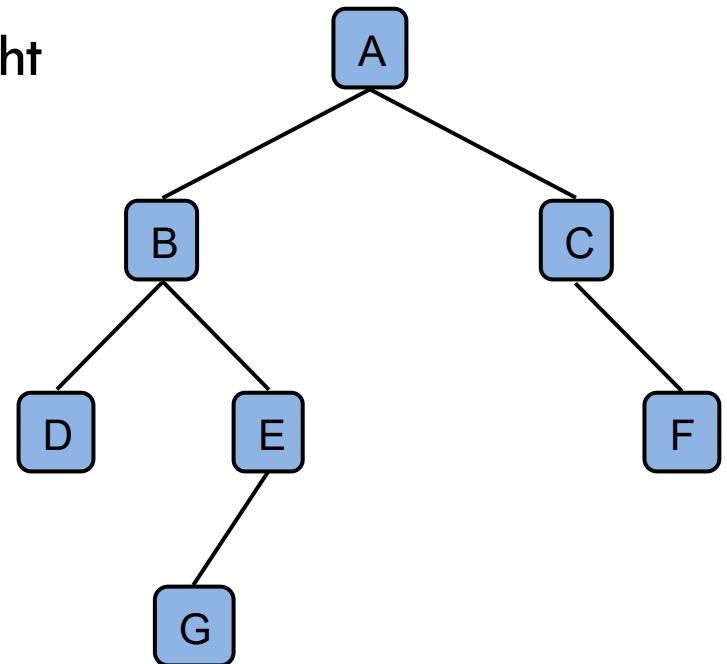
Binary Trees

A **binary tree** is an ordered tree with the following properties:

- Each internal node has at most two children
- Each child node is labeled as a **left child** or a **right child**
- Child ordering is left followed by right

The right/left subtree is the subtree root at the right/left child.

We say the tree is **proper** if every internal node has two children

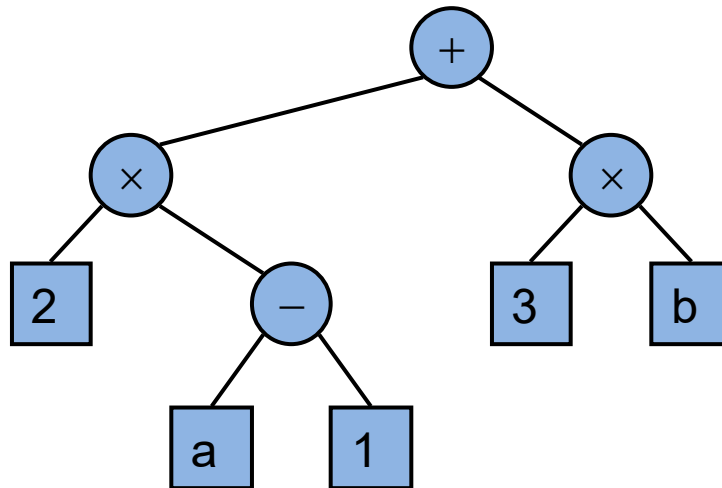


Binary tree application: Arithmetic expression tree

Binary tree associated with an arithmetic expression

- internal nodes: operators
- external nodes: operands

Example: Arithmetic expression tree for $(2 \times (a - 1) + (3 \times b))$

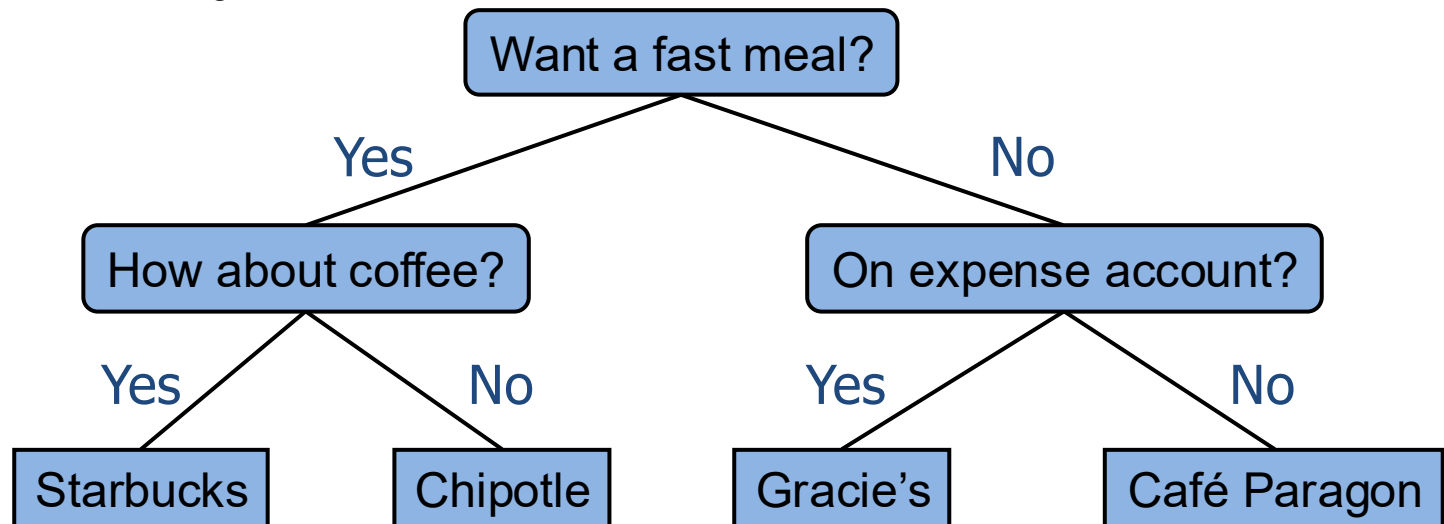


Binary tree application: Decision trees

Tree associated with a decision process

- internal nodes: questions with yes/no answer
- external nodes: decisions

Example: dining decision



Binary Tree Operations

- A **binary tree** extends the Tree operations, i.e., it inherits all the methods of a tree.
- Update methods may be defined by data structures implementing the binary tree
- Additional methods:
 - position **leftChild**(p)
 - position **rightChild**(p)
 - position **sibling**(p)

return null when there is no left, right, or sibling of p, respectively

Node object

Node object implementation typically has the following attributes:

- value: the value associated with this Node
- left: left child of this Node
- right: right child of this Node
- parent: (optional) the parent of this Node

```
def is_external(v)  
    # test if v is a leaf  
    return v.left = null and v.right = null
```

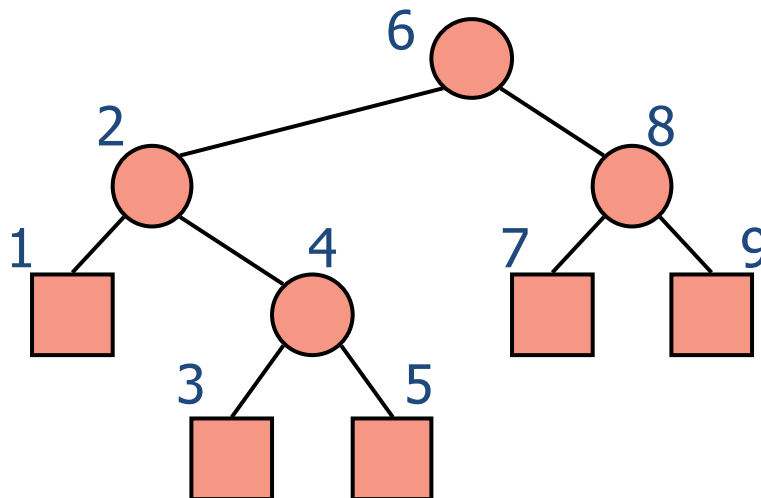
Inorder Traversal

To do an inorder traversal starting at a given node, the node is visited after its left subtree but before its right subtree

Visit does some work on the node:

- print node data
- aggregate node data
- modify node data

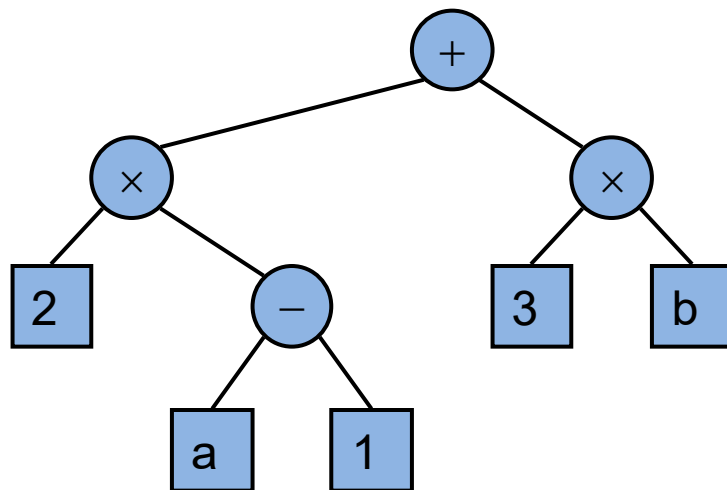
```
def in_order(v)
    if v.left ≠ null then
        in_order(v.left)
    visit(v)
    if v.right ≠ null then
        in_order(v.right)
```



Print Arithmetic Expressions

Extended inorder traversal:

- print operand or operator when visiting node
- print “(“ before left subtree
- print “)” after right subtree



```
def print_expr(v)
    if v.left ≠ null then
        print("(")
        print_expr(v.left)
    print(v.element)
    if v.right ≠ null then
        print_expr(v.right)
    print(")")
```

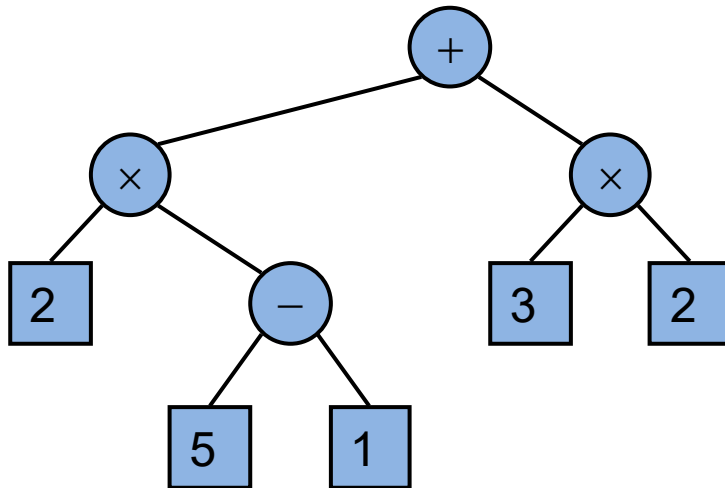
$((2 \times (a - 1)) + (3 \times b))$

Evaluate Arithmetic Expressions

Extended postorder traversal

- recursive method returning the value of a subtree
- when visiting an internal node, combine the values of the subtrees

```
def eval_expr(v)
  if v.is_external() then
    return v.element
  else
    x ← eval_expr(v.left)
    y ← eval_expr(v.right)
     $\oplus$  ← v.element
    return x  $\oplus$  y
```

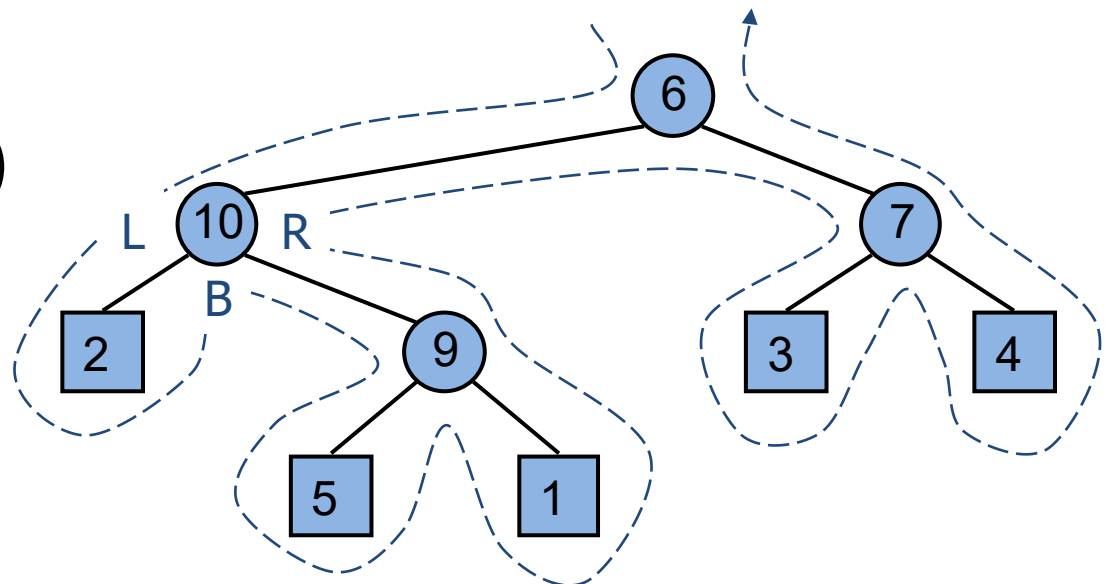


Euler Tour Traversal

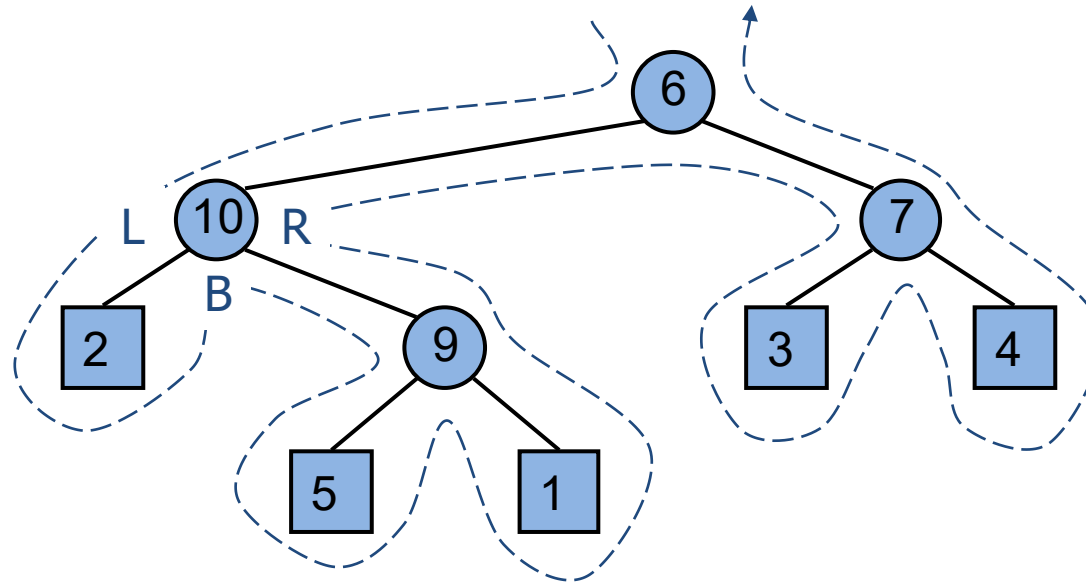
Generic traversal of a binary tree. Includes as special cases the preorder, postorder and inorder traversals

Walk around the tree, keeping the tree on your left, and visit each node three times:

- on the left (preorder)
- from below (inorder)
- on the right (postorder)



Euler Tour Traversal



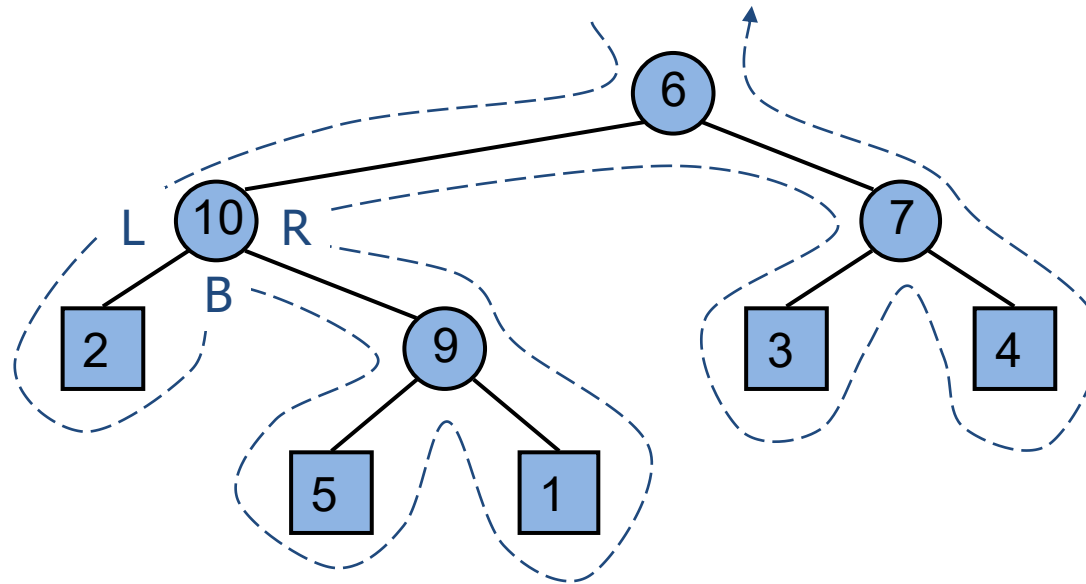
6,10,2,2,2,10,9,5,5,5,9,1,1,1,9,10,6,7,3,3,3,7,4,4,4,7,6

Preorder (first visit):

Inorder (second visit):

Postorder (third visit):

Euler Tour Traversal



6,10,2,2,10,9,5,5,5,9,1,1,1,9,10,6,7,3,3,3,7,4,4,4,7,6

Preorder (**first visit**): 6, 10, 2, 9, 5, 1, 7, 3, 4

Inorder (**second visit**): 2, 10, 5, 9, 1, 6, 3, 7, 4

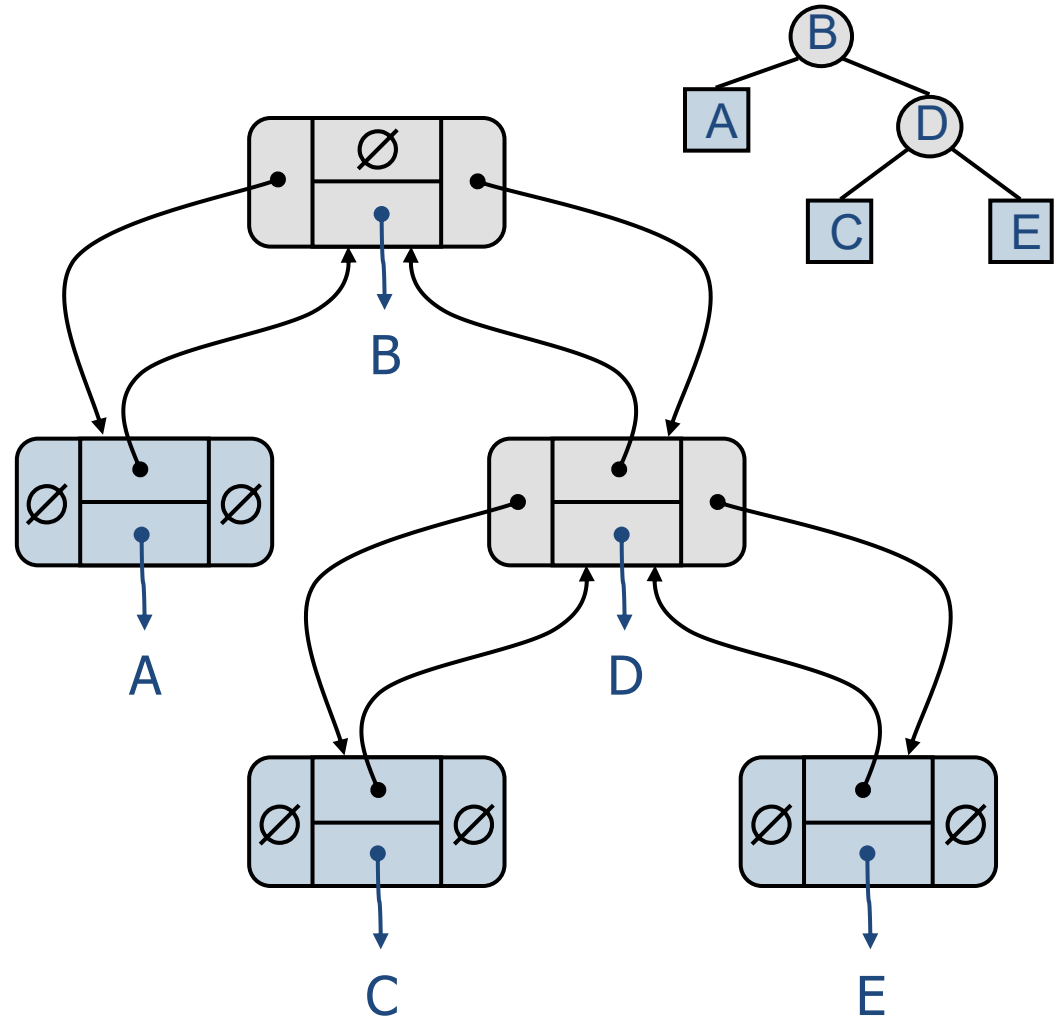
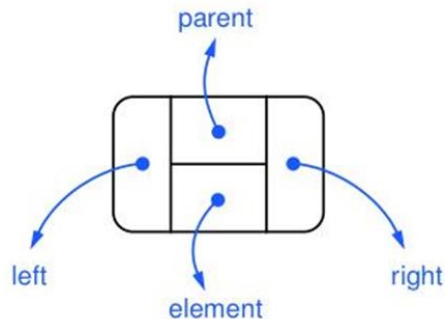
Postorder (**third visit**): 2, 5, 1, 9, 10, 3, 4, 7, 6

Linked Structure for Binary Trees

A node is represented by an object storing

- Element
- Parent node
- Left child node
- Right child node

Node objects implement the Position ADT

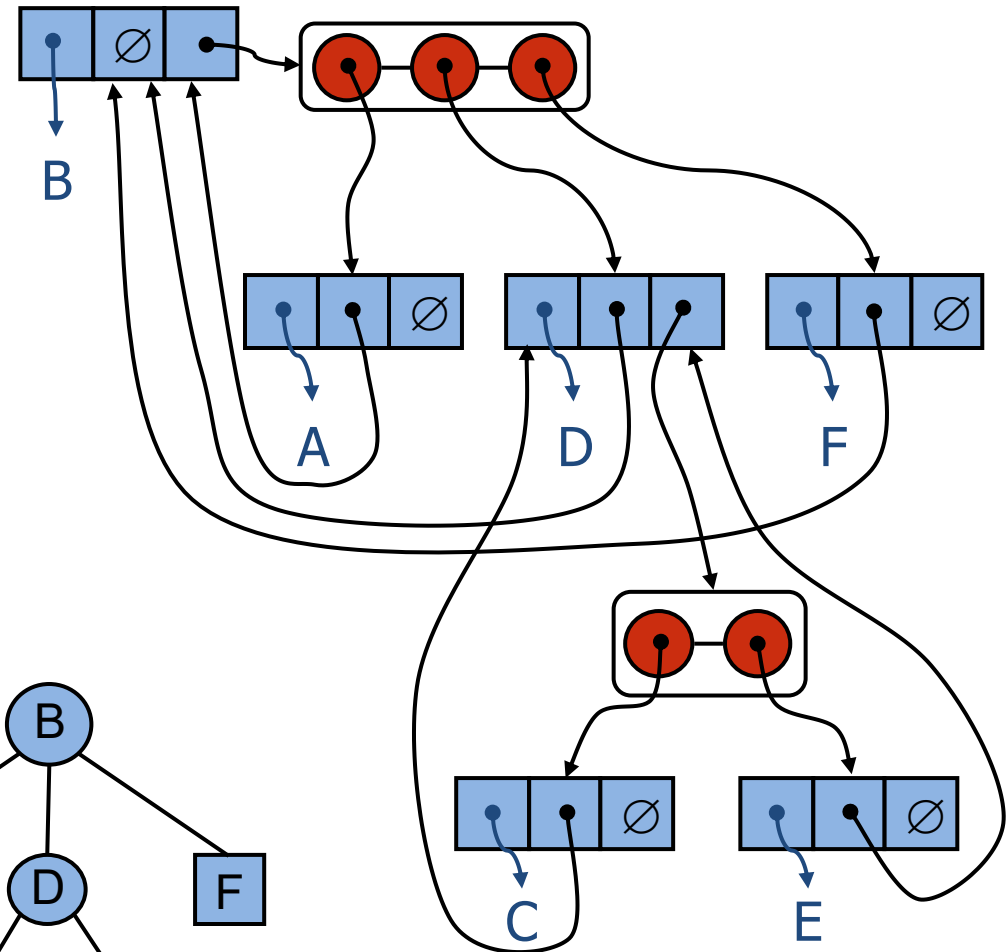
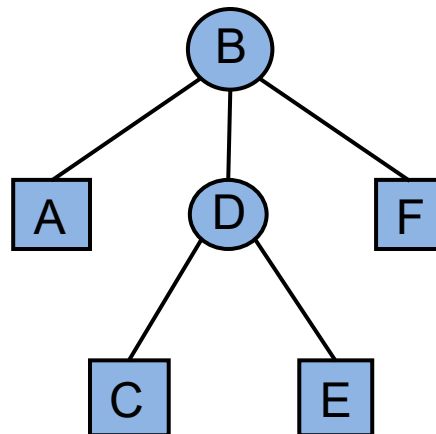
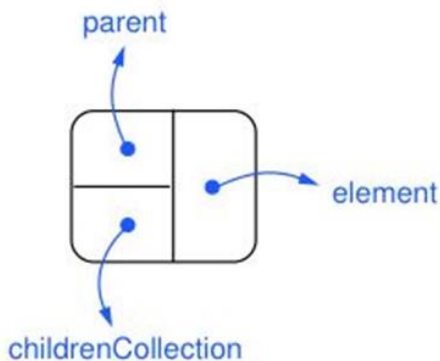


Linked Structure for General Trees

A node is represented by an object storing

- Element
- Parent node
- Sequence of children

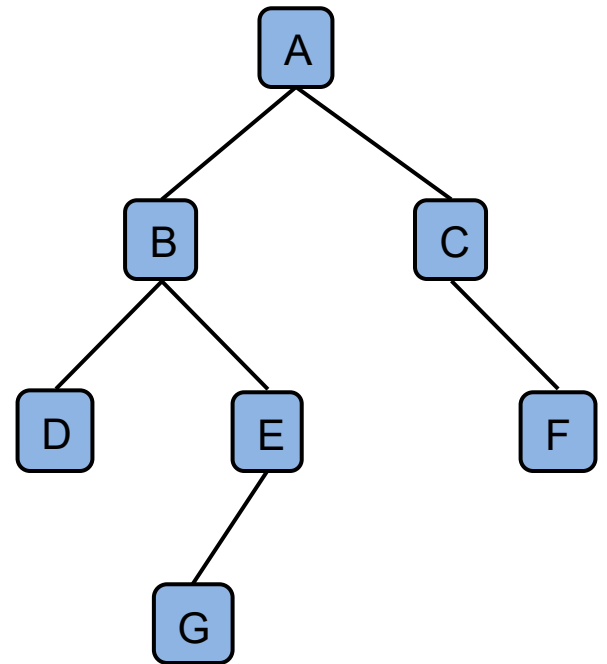
Node objects implement the Position ADT



Examples of recursive code on trees

```
def depth(v)
    # compute the depth of v

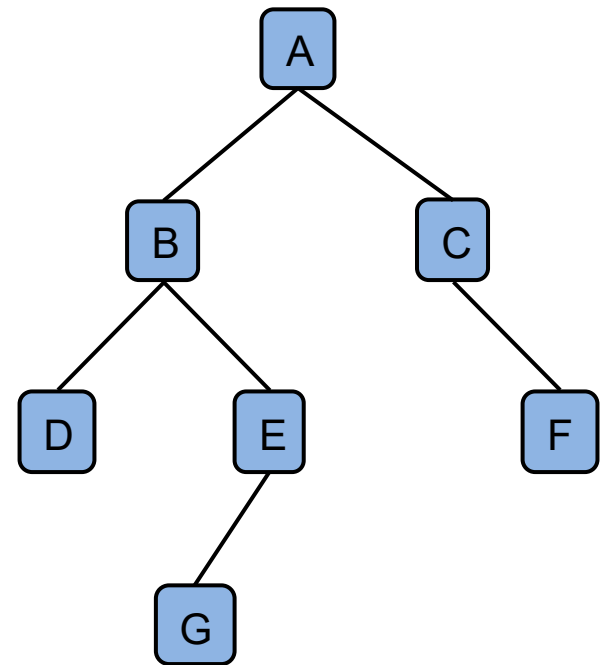
    if v.parent = null then
        # root's depth is 0
        return 0
    else
        return depth(v.parent) + 1.
```



Examples of recursive code on trees

```
def height(v)
    # compute height of subtree at v

    if v.isExternal() then
        # a leaf's height is 0
        return 0
    else
        h ← 0
        for each child w of v
            h ← max(h, height(w))
        return h + 1
```



Complexity analysis of recursive algorithms on trees

Sometimes, the method may call itself on all children

- In worst case, do a call on every node
- If the work done, *excluding the recursion*, is constant per call, then the total cost is **linear in the number of nodes**

Sometimes, the method calls itself on at most one child

- In worst case, do one call at each level of the tree
- If the work done, *excluding the recursion*, is constant per call, then the total cost is **linear in the height of the tree**

Binary Search Tree

So far we've been focused on the structure of the tree. The real usefulness of trees hinges on the values we store at each element and how these values are laid out.

BST is a data structure for storing values that can be sorted. These values are laid out so that an in-order traversal of the BST visits the values in sorted order.

Can search for elements and insert/delete operations run in $O(\log n)$ time provided the tree is “balanced”. More on that next week!

